PYTHON ACTIVITY BASED LEARNING

10 Complete Case Studies | Algorithms, Flowcharts, Pseudocode, Code and Tests

All example rates, limits, priorities and datasets are illustrative and can be adjusted to match assignment rules.

Case Study 2: Progressive Electricity Billing

1. Problem Statement

Calculate monthly progressive slab bills, fixed charges, surcharges, rebates and unusual increases.

2. Objectives

Compute bills accurately; compare monthly spending; flag increases over 25%.

3. Assumptions

Slabs: first 100 at Rs 2, next 100 at Rs 3, next 100 at Rs 5, remaining at Rs 7. Fixed Rs 50; surcharge 5% above 300 units; rebate 10% at 100 units or less.

4. Algorithm

1. Start. 2. Read or initialize input values. 3. Validate input and stored data. 4. Use loops to process each item. 5. Apply conditions and compute results. 6. Display summary and decisions. 7. Stop.

5. Flowchart

```
[START]
   |
[Input / Initialize]
   |
<Valid input?> -- No --> [Error / Reject]
   | Yes
[Loop through records]
   |
[Calculate / Compare / Update]
   |
[Display results and summary]
   |
 [END]
```

6. Pseudocode

```
BEGIN
  READ or INITIALIZE inputs
  FOR each relevant record
    IF invalid THEN reject or stop
    ELSE calculate values and update lists
  END FOR
  COMPARE results with rules or thresholds
  DISPLAY report
END
```

7. Python Source Code

```
months = int(input("Number of months: "))
bills = []
for i in range(months):
    u = int(input("Units for month " + str(i+1) + ": "))
    if u < 0:
        print("Invalid units")
        break
    charge = min(u,100)*2 + max(0,min(u-100,100))*3
    charge += max(0,min(u-200,100))*5 + max(0,u-300)*7
    surcharge = charge*0.05 if u>300 else 0
    rebate = charge*0.10 if u<=100 else 0
    bill = charge+50+surcharge-rebate
    bills.append(bill)
```

```
    print("Bill: Rs",round(bill,2))
if len(bills)==months and months>0:
    for i in range(1,months):
        if bills[i]>bills[i-1]*1.25:
            print("Unusual increase in month",i+1)
    print("Total:",round(sum(bills),2))
    print("Average:",round(sum(bills)/months,2))
```

8. Test Cases

Test 1 (normal case): 3 months: 80, 150, 250 units. Test 2 (boundary case): use zero, minimum or exact threshold values where applicable. Test 3 (invalid case): try an out-of-range value and verify the program rejects it where validation is provided.

9. Output Screenshots (sample console-output mockup)

```
SAMPLE RUN / EXPECTED OUTPUT
Bills: 194.0, 400.0, 850.0; unusual increases in months 2 and 3; total
1444.0.
```

10. Conclusion

The program demonstrates syllabus-level Python techniques to process the case study, apply rules and produce a clear result. The sample output is illustrative; execute the code in VS Code or Python IDLE to capture authentic screenshots for submission.

Case Study 4: Student Feedback Sentiment Scoring

1. Problem Statement

Score feedback sentences using positive and negative keyword lists.

2. Objectives

Count indicators, calculate sentiment scores and summarize classifications.

3. Assumptions

Each keyword occurrence is counted as a whole word; score = positives minus negatives; thresholds +/-2 for very positive/negative.

4. Algorithm

1. Start. 2. Read or initialize input values. 3. Validate input and stored data. 4. Use loops to process each item. 5. Apply conditions and compute results. 6. Display summary and decisions. 7. Stop.

5. Flowchart

```
[START]
   |
[Input / Initialize]
   |
<Valid input?> -- No --> [Error / Reject]
   | Yes
[Loop through records]
   |
[Calculate / Compare / Update]
   |
[Display results and summary]
   |
 [END]
```

6. Pseudocode

```
BEGIN
  READ or INITIALIZE inputs
  FOR each relevant record
    IF invalid THEN reject or stop
    ELSE calculate values and update lists
  END FOR
  COMPARE results with rules or thresholds
  DISPLAY report
END
```

7. Python Source Code

```
positive = ["good","excellent","helpful","interesting","great"]
negative = ["bad","poor","boring","difficult","worst"]
n = int(input("Number of feedback sentences: "))
counts = [0,0,0]
for i in range(n):
    sentence = input("Feedback: ").lower()
    for mark in ",.!?;:":
        sentence = sentence.replace(mark," ")
    words = sentence.split()
    p = 0
    q = 0
    for word in words:
        if word in positive: p += 1
```

```
        if word in negative: q += 1
    score = p-q
    if score >= 2: label = "Very Positive"
    elif score == 1: label = "Positive"
    elif score == 0: label = "Neutral"
    elif score == -1: label = "Negative"
    else: label = "Very Negative"
    if score>0: counts[0]+=1
    elif score<0: counts[1]+=1
    else: counts[2]+=1
    print("Positive:",p,"Negative:",q,"Score:",score,label)
print("Positive/Negative/Neutral totals:",counts)
```

8. Test Cases

Test 1 (normal case): good helpful / boring difficult / interesting. Test 2 (boundary case): use zero, minimum or exact threshold values where applicable. Test 3 (invalid case): try an out-of-range value and verify the program rejects it where validation is provided.

9. Output Screenshots (sample console-output mockup)

```
SAMPLE RUN / EXPECTED OUTPUT
Scores: 2 Very Positive; -2 Very Negative; 1 Positive; summary [2, 1, 0].
```

10. Conclusion

The program demonstrates syllabus-level Python techniques to process the case study, apply rules and produce a clear result. The sample output is illustrative; execute the code in VS Code or Python IDLE to capture authentic screenshots for submission.

Case Study 8: Manufacturing Cost and Profitability Analysis

1. Problem Statement

Compute manufacturing cost and profit against target.

2. Objectives

Include material wastage, labour, machines and selling prices; evaluate targets.

3. Assumptions

Wastage increases material cost by the supplied percent; profit percent uses total cost as denominator.

4. Algorithm

1. Start. 2. Read or initialize input values. 3. Validate input and stored data. 4. Use loops to process each item. 5. Apply conditions and compute results. 6. Display summary and decisions. 7. Stop.

5. Flowchart

```
[START]
   |
[Input / Initialize]
   |
<Valid input?> -- No --> [Error / Reject]
   | Yes
[Loop through records]
   |
[Calculate / Compare / Update]
   |
[Display results and summary]
   |
 [END]
```

6. Pseudocode

```
BEGIN
  READ or INITIALIZE inputs
  FOR each relevant record
    IF invalid THEN reject or stop
    ELSE calculate values and update lists
  END FOR
  COMPARE results with rules or thresholds
  DISPLAY report
END
```

7. Python Source Code

```
n = int(input("Products: "))
overall = 0
for i in range(n):
    name = input("Product: ")
    qty = int(input("Quantity: "))
    material = float(input("Material cost/unit: "))
    labour = float(input("Labour cost/unit: "))
    machine = float(input("Machine cost/unit: "))
    waste = float(input("Wastage percent: "))
    selling = float(input("Selling price/unit: "))
    target = float(input("Target profit percent: "))
    if qty<=0 or min(material,labour,machine,waste,selling,target)<0:
```

```
        print("Invalid input")
        break
    cost = qty*(material*(1+waste/100)+labour+machine)
    revenue = qty*selling
    profit = revenue-cost
    percent = profit/cost*100 if cost>0 else 0
    overall += profit
    print(name,"Cost:",round(cost,2),"Revenue:",round(revenue,2))
    print("Profit:",round(profit,2),"Percent:",round(percent,2))
    print("Target achieved" if percent>=target and profit>0 else "Target not
achieved")
else:
    print("Overall profit/loss:",round(overall,2))
```

8. Test Cases

Test 1 (normal case): Chair; 100 units; material 500, labour 100, machine 50, waste 10%, price 800, target 20%. Test 2 (boundary case): use zero, minimum or exact threshold values where applicable. Test 3 (invalid case): try an out-of-range value and verify the program rejects it where validation is provided.

9. Output Screenshots (sample console-output mockup)

```
SAMPLE RUN / EXPECTED OUTPUT
Cost 70000; revenue 80000; profit 10000; margin on cost 14.29%; target not
achieved.
```

10. Conclusion

The program demonstrates syllabus-level Python techniques to process the case study, apply rules and produce a clear result. The sample output is illustrative; execute the code in VS Code or Python IDLE to capture authentic screenshots for submission.

Case Study 9: Multi-Currency Travel Budget Analysis

1. Problem Statement

Convert expenses into INR and compare total spending with a budget.

2. Objectives

Apply exchange rates and fees; find largest expense category.

3. Assumptions

Exchange rate is INR per foreign unit; a single fee percentage applies to every expense.

4. Algorithm

1. Start. 2. Read or initialize input values. 3. Validate input and stored data. 4. Use loops to process each item. 5. Apply conditions and compute results. 6. Display summary and decisions. 7. Stop.

5. Flowchart

```
[START]
   |
[Input / Initialize]
   |
<Valid input?> -- No --> [Error / Reject]
   | Yes
[Loop through records]
   |
[Calculate / Compare / Update]
   |
[Display results and summary]
   |
 [END]
```

6. Pseudocode

```
BEGIN
  READ or INITIALIZE inputs
  FOR each relevant record
    IF invalid THEN reject or stop
    ELSE calculate values and update lists
  END FOR
  COMPARE results with rules or thresholds
  DISPLAY report
END
```

7. Python Source Code

```
n = int(input("Expense categories: "))
budget = float(input("Budget INR: "))
fee = float(input("Conversion fee percent: "))
names = []
amounts = []
if n>0 and budget>=0 and fee>=0:
    for i in range(n):
        name = input("Category: ")
        foreign = float(input("Foreign amount: "))
        rate = float(input("INR per foreign unit: "))
        if foreign<0 or rate<=0:
            print("Invalid expense")
            break
        value = foreign*rate*(1+fee/100)
        names.append(name)
```

```
        amounts.append(value)
        print(name,"INR",round(value,2))
    if len(amounts)==n:
        total = sum(amounts)
        biggest = 0
        for i in range(1,n):
            if amounts[i]>amounts[biggest]: biggest=i
        print("Total:",round(total,2))
        print("Largest:",names[biggest],round(amounts[biggest],2))
        print("Over budget" if total>budget else "Within budget")
        print("Difference:",round(abs(budget-total),2))
```

8. Test Cases

Test 1 (normal case): Budget 50000; fee 2%; Hotel 200x85; Food 100x85; Shopping 300x85. Test 2 (boundary case): use zero, minimum or exact threshold values where applicable. Test 3 (invalid case): try an out-of-range value and verify the program rejects it where validation is provided.

9. Output Screenshots (sample console-output mockup)

```
SAMPLE RUN / EXPECTED OUTPUT
Expenses 17340, 8670, 26010; total 52020; Shopping largest; over budget by
2020.
```

10. Conclusion

The program demonstrates syllabus-level Python techniques to process the case study, apply rules and produce a clear result. The sample output is illustrative; execute the code in VS Code or Python IDLE to capture authentic screenshots for submission.

Case Study 11: Examination Result and Grace-Mark Validation

1. Problem Statement

Evaluate subject results with limited grace marks.

2. Objectives

Validate marks, allocate grace, classify final result.

3. Assumptions

Same maximum and passing mark for all subjects; grace is a shared pool; classification 75/60/50 percent.

4. Algorithm

1. Start. 2. Read or initialize input values. 3. Validate input and stored data. 4. Use loops to process each item. 5. Apply conditions and compute results. 6. Display summary and decisions. 7. Stop.

5. Flowchart

```
[START]
   |
[Input / Initialize]
   |
<Valid input?> -- No --> [Error / Reject]
   | Yes
[Loop through records]
   |
[Calculate / Compare / Update]
   |
[Display results and summary]
   |
 [END]
```

6. Pseudocode

```
BEGIN
  READ or INITIALIZE inputs
  FOR each relevant record
    IF invalid THEN reject or stop
    ELSE calculate values and update lists
  END FOR
  COMPARE results with rules or thresholds
  DISPLAY report
END
```

7. Python Source Code

```
n = int(input("Subjects: "))
maximum = int(input("Maximum per subject: "))
passing = int(input("Passing mark: "))
limit = int(input("Total grace allowed: "))
marks = []
if n>0 and maximum>0 and 0<=passing<=maximum and limit>=0:
    for i in range(n):
        m = int(input("Subject "+str(i+1)+" marks: "))
        if m<0 or m>maximum:
            print("Invalid marks")
            break
        marks.append(m)
```

```
if len(marks)==n:
    final = []
    used = 0
    for m in marks:
        needed = max(0,passing-m)
        if needed<=limit-used:
            m += needed
            used += needed
        final.append(m)
    total = sum(final)
    percent = total/(n*maximum)*100
    if min(final)<passing: result = "FAIL"
    elif percent>=75: result = "DISTINCTION"
    elif percent>=60: result = "FIRST CLASS"
    elif percent>=50: result = "SECOND CLASS"
    else: result = "PASS"
    print("Final marks:",final,"Grace:",used)
    print("Total:",total,"Percentage:",round(percent,2),result)
```

8. Test Cases

Test 1 (normal case): 3 subjects; maximum 100, passing 35, grace 5; marks 32, 75, 80. Test 2 (boundary case): use zero, minimum or exact threshold values where applicable. Test 3 (invalid case): try an out-of-range value and verify the program rejects it where validation is provided.

9. Output Screenshots (sample console-output mockup)

```
SAMPLE RUN / EXPECTED OUTPUT
Final marks [35,75,80]; grace 3; total 190; 63.33%; FIRST CLASS.
```

10. Conclusion

The program demonstrates syllabus-level Python techniques to process the case study, apply rules and produce a clear result. The sample output is illustrative; execute the code in VS Code or Python IDLE to capture authentic screenshots for submission.

Case Study 26: Nested Loop Examination Seating Generator

1. Problem Statement

Produce a classroom seating grid with reserved seats and alternate empty positions.

2. Objectives

Respect classroom size, identify seats and report counts.

3. Assumptions

Seats with even row+column sum are left empty unless reserved; reserved seats take priority.

4. Algorithm

1. Start. 2. Read or initialize input values. 3. Validate input and stored data. 4. Use loops to process each item. 5. Apply conditions and compute results. 6. Display summary and decisions. 7. Stop.

5. Flowchart

```
[START]
   |
[Input / Initialize]
   |
<Valid input?> -- No --> [Error / Reject]
   | Yes
[Loop through records]
   |
[Calculate / Compare / Update]
   |
[Display results and summary]
   |
 [END]
```

6. Pseudocode

```
BEGIN
  READ or INITIALIZE inputs
  FOR each relevant record
    IF invalid THEN reject or stop
    ELSE calculate values and update lists
  END FOR
  COMPARE results with rules or thresholds
  DISPLAY report
END
```

7. Python Source Code

```
rows = int(input("Rows: "))
cols = int(input("Columns: "))
reserved = []
if rows>0 and cols>0:
    k = int(input("Reserved positions: "))
    for i in range(k):
        r = int(input("Reserved row: "))
        c = int(input("Reserved column: "))
        if r<1 or r>rows or c<1 or c>cols or (r,c) in reserved:
            print("Invalid/duplicate reserved seat")
            break
        reserved.append((r,c))
```

```
if len(reserved)==k:
    student = 0
    empty = 0
    for r in range(1,rows+1):
        line = []
        for c in range(1,cols+1):
            if (r,c) in reserved: seat = "RES"
            elif (r+c)%2==0:
                seat = "---"
                empty += 1
            else:
                student += 1
                seat = "S"+str(student)
            line.append(seat)
        print("  ".join(line))
    print("Capacity:",rows*cols,"Reserved:",k)
    print("Students:",student,"Empty:",empty)
```

8. Test Cases

Test 1 (normal case): 4 rows, 4 columns; reserved (1,1), (2,2). Test 2 (boundary case): use zero, minimum or exact threshold values where applicable. Test 3 (invalid case): try an out-of-range value and verify the program rejects it where validation is provided.

9. Output Screenshots (sample console-output mockup)

```
SAMPLE RUN / EXPECTED OUTPUT
RES S1 --- S2 / S3 RES S4 --- / --- S5 --- S6 / S7 --- S8 ---; 8 students, 6
empty.
```

10. Conclusion

The program demonstrates syllabus-level Python techniques to process the case study, apply rules and produce a clear result. The sample output is illustrative; execute the code in VS Code or Python IDLE to capture authentic screenshots for submission.

Case Study 48: Tuple-Based Employee Shift Conflict Detection

1. Problem Statement

Detect overlapping assignments for the same employee.

2. Objectives

Store shifts as tuples; compare time intervals; list employees with conflicts.

3. Assumptions

Same-day 24-hour clock; touching shifts are not overlapping; overnight shifts excluded.

4. Algorithm

1. Start. 2. Read or initialize input values. 3. Validate input and stored data. 4. Use loops to process each item. 5. Apply conditions and compute results. 6. Display summary and decisions. 7. Stop.

5. Flowchart

```
[START]
   |
[Input / Initialize]
   |
<Valid input?> -- No --> [Error / Reject]
   | Yes
[Loop through records]
   |
[Calculate / Compare / Update]
   |
[Display results and summary]
   |
 [END]
```

6. Pseudocode

```
BEGIN
  READ or INITIALIZE inputs
  FOR each relevant record
    IF invalid THEN reject or stop
    ELSE calculate values and update lists
  END FOR
  COMPARE results with rules or thresholds
  DISPLAY report
END
```

7. Python Source Code

```
n = int(input("Number of shifts: "))
shifts = []
for i in range(n):
    name = input("Employee: ")
    start = int(input("Start hour (0-23): "))
    end = int(input("End hour (1-24): "))
    if start<0 or start>23 or end<=start or end>24:
        print("Invalid shift")
        break
    shifts.append((name,start,end))
if len(shifts)==n:
    conflicts = []
```

```
employees = []
for i in range(n):
    for j in range(i+1,n):
        name1,s1,e1 = shifts[i]
        name2,s2,e2 = shifts[j]
        if name1==name2 and s1<e2 and s2<e1:
            conflicts.append((i+1,j+1,name1))
            if name1 not in employees: employees.append(name1)
for a,b,name in conflicts:
    print("Conflict:",name,"shifts",a,"and",b)
print("Total conflicts:",len(conflicts))
print("Employees with conflicts:",employees)
```

8. Test Cases

Test 1 (normal case): Ravi 9-14; Ravi 12-17; Priya 10-15. Test 2 (boundary case): use zero, minimum or exact threshold values where applicable. Test 3 (invalid case): try an out-of-range value and verify the program rejects it where validation is provided.

9. Output Screenshots (sample console-output mockup)

```
SAMPLE RUN / EXPECTED OUTPUT
Conflict: Ravi shifts 1 and 2; total conflicts 1; employees [Ravi].
```

10. Conclusion

The program demonstrates syllabus-level Python techniques to process the case study, apply rules and produce a clear result. The sample output is illustrative; execute the code in VS Code or Python IDLE to capture authentic screenshots for submission.

Case Study 57: Smart Canteen Order and Stock Management

1. Problem Statement

Process customer orders while maintaining canteen inventory.

2. Objectives

Calculate bills, reject insufficient stock, summarize daily sales.

3. Assumptions

Each order contains one menu item; no negative quantities; rejected orders do not change stock.

4. Algorithm

1. Start. 2. Read or initialize input values. 3. Validate input and stored data. 4. Use loops to process each item. 5. Apply conditions and compute results. 6. Display summary and decisions. 7. Stop.

5. Flowchart

```
[START]
   |
[Input / Initialize]
   |
<Valid input?> -- No --> [Error / Reject]
   | Yes
[Loop through records]
   |
[Calculate / Compare / Update]
   |
[Display results and summary]
   |
 [END]
```

6. Pseudocode

```
BEGIN
  READ or INITIALIZE inputs
  FOR each relevant record
    IF invalid THEN reject or stop
    ELSE calculate values and update lists
  END FOR
  COMPARE results with rules or thresholds
  DISPLAY report
END
```

7. Python Source Code

```
items = ["Idli","Dosa","Meals","Juice","Tea"]
prices = [30,50,80,40,15]
stock = [50,40,30,25,60]
sales = 0
successful = 0
for i in range(len(items)):
    print(i+1,items[i],"Rs",prices[i],"Stock",stock[i])
n = int(input("Customer orders: "))
for i in range(n):
    item = int(input("Item number: "))
    qty = int(input("Quantity: "))
    if item<1 or item>len(items) or qty<=0:
```

```
        print("Invalid order")
    elif qty>stock[item-1]:
        print("Rejected: only",stock[item-1],"available")
    else:
        idx = item-1
        bill = prices[idx]*qty
        stock[idx] -= qty
        sales += bill
        successful += 1
        print("Accepted:",items[idx],"Bill Rs",bill)
print("Successful orders:",successful,"Total sales Rs",sales)
for i in range(len(items)):
    print(items[i],"remaining:",stock[i])
```

8. Test Cases

Test 1 (normal case): Orders: Idli x5, Dosa x3, Meals x35. Test 2 (boundary case): use zero, minimum or exact threshold values where applicable. Test 3 (invalid case): try an out-of-range value and verify the program rejects it where validation is provided.

9. Output Screenshots (sample console-output mockup)

```
SAMPLE RUN / EXPECTED OUTPUT
Accepted 150, accepted 150, Meals rejected; sales 300; Idli 45, Dosa 37,
Meals 30.
```

10. Conclusion

The program demonstrates syllabus-level Python techniques to process the case study, apply rules and produce a clear result. The sample output is illustrative; execute the code in VS Code or Python IDLE to capture authentic screenshots for submission.

Case Study 78: Hospital Resource Allocation and Capacity Monitoring

1. Problem Statement

Analyze ward bed occupancy, staffing and equipment availability.

2. Objectives

Detect resource shortages and rank wards using simple supplied rules.

3. Assumptions

Critical bed threshold is 2 or fewer available; each of three shortage types adds one priority point. Not clinical triage.

4. Algorithm

1. Start. 2. Read or initialize input values. 3. Validate input and stored data. 4. Use loops to process each item. 5. Apply conditions and compute results. 6. Display summary and decisions. 7. Stop.

5. Flowchart

```
[START]
   |
[Input / Initialize]
   |
<Valid input?> -- No --> [Error / Reject]
   | Yes
[Loop through records]
   |
[Calculate / Compare / Update]
   |
[Display results and summary]
   |
 [END]
```

6. Pseudocode

```
BEGIN
  READ or INITIALIZE inputs
  FOR each relevant record
    IF invalid THEN reject or stop
    ELSE calculate values and update lists
  END FOR
  COMPARE results with rules or thresholds
  DISPLAY report
END
```

7. Python Source Code

```
wards = ["Emergency","General","ICU","Maternity"]
# occupied,total beds,staff,needed staff,equipment,needed equipment
resources = [[18,20,3,5,4,5],[35,40,8,7,10,8],
             [9,10,4,6,3,5],[12,15,4,4,6,5]]
priorities = []
for i in range(len(wards)):
    occupied,total,staff,need_staff,equip,need_equip = resources[i]
    available = total-occupied
    score = 0
    if available<=2: score+=1
    if staff<need_staff: score+=1
```

```
    if equip<need_equip: score+=1
    priorities.append(score)
    if score>=2: label="HIGH"
    elif score==1: label="MEDIUM"
    else: label="LOW"
    print(wards[i],"Available beds:",available,
          "Occupancy %:",round(occupied/total*100,2),
          "Shortages:",score,"Priority:",label)
most = max(priorities)
print("Highest shortage count:",most)
for i in range(len(wards)):
    if priorities[i]==most and most>0:
        print("Attention:",wards[i])
```

8. Test Cases

Test 1 (normal case): Four sample wards: Emergency, General, ICU, Maternity. Test 2 (boundary case): use zero, minimum or exact threshold values where applicable. Test 3 (invalid case): try an out-of-range value and verify the program rejects it where validation is provided.

9. Output Screenshots (sample console-output mockup)

```
SAMPLE RUN / EXPECTED OUTPUT
Emergency HIGH (3), General LOW (0), ICU HIGH (3), Maternity LOW (0).
```

10. Conclusion

The program demonstrates syllabus-level Python techniques to process the case study, apply rules and produce a clear result. The sample output is illustrative; execute the code in VS Code or Python IDLE to capture authentic screenshots for submission.

Case Study 92: Multi-Event Sports Scheduling and Scoring

1. Problem Statement

Identify event venue conflicts and rank teams by combined points.

2. Objectives

Represent schedules with tuples and scores with nested lists; total and sort results.

3. Assumptions

Events conflict only when they overlap at the same venue; ranking uses descending total points.

4. Algorithm

1. Start. 2. Read or initialize input values. 3. Validate input and stored data. 4. Use loops to process each item. 5. Apply conditions and compute results. 6. Display summary and decisions. 7. Stop.

5. Flowchart

```
[START]
   |
[Input / Initialize]
   |
<Valid input?> -- No --> [Error / Reject]
   | Yes
[Loop through records]
   |
[Calculate / Compare / Update]
   |
[Display results and summary]
   |
 [END]
```

6. Pseudocode

```
BEGIN
  READ or INITIALIZE inputs
  FOR each relevant record
    IF invalid THEN reject or stop
    ELSE calculate values and update lists
  END FOR
  COMPARE results with rules or thresholds
  DISPLAY report
END
```

7. Python Source Code

```
teams = ["Team A","Team B","Team C","Team D"]
scores = [[10,20,15],[25,10,20],[15,25,10],[20,15,25]]
schedules = [("Football",9,11,"Ground A"),
             ("Basketball",10,12,"Ground B"),
             ("Volleyball",12,14,"Ground A"),
             ("Cricket",14,16,"Ground C")]
conflicts = 0
for i in range(len(schedules)):
    for j in range(i+1,len(schedules)):
        a,s1,e1,v1 = schedules[i]
        b,s2,e2,v2 = schedules[j]
        if v1==v2 and s1<e2 and s2<e1:
```

```
            print("Conflict:",a,b)
            conflicts+=1
print("Venue conflicts:",conflicts)
totals = []
for row in scores:
    total = 0
    for points in row: total+=points
    totals.append(total)
for i in range(len(teams)):
    best=i
    for j in range(i+1,len(teams)):
        if totals[j]>totals[best]: best=j
    totals[i],totals[best] = totals[best],totals[i]
    teams[i],teams[best] = teams[best],teams[i]
for i in range(len(teams)):
    print("Rank",i+1,teams[i],"Points",totals[i])
```

8. Test Cases

Test 1 (normal case): Sample schedules and scores shown in source code. Test 2 (boundary case): use zero, minimum or exact threshold values where applicable. Test 3 (invalid case): try an out-of-range value and verify the program rejects it where validation is provided.

9. Output Screenshots (sample console-output mockup)

```
SAMPLE RUN / EXPECTED OUTPUT
No venue conflicts; ranks Team D 60, Team B 55, Team C 50, Team A 45.
```

10. Conclusion

The program demonstrates syllabus-level Python techniques to process the case study, apply rules and produce a clear result. The sample output is illustrative; execute the code in VS Code or Python IDLE to capture authentic screenshots for submission.